Analisi Dashboard

November 30, 2025

1 Analisi Interattiva di un Dataset di Vendite

In questo progetto didattico verrà analizzato un dataset simulato di vendite, composto da ordini effettuati in diverse regioni del mondo e per differenti categorie di prodotti. L'obiettivo principale è esplorare i dati attraverso la produzione di grafici interattivi, che permetteranno di visualizzare trend, confrontare performance tra categorie e regioni.

Il progetto è suddiviso come segue:

1. Import Librerie, Matplotlib Settings e Funzioni Utili:
2. Generazione Dataset
3. Pulizia e preparazione del dataset
4. Analisi esplorativa dei dati (EDA)
5. Visualizzazione interattiva (matplotlib, plotly, ipywidgets)

1.1 Import Librerie, Matplotlib Settings e Funzioni Utili

[ ]:
```
## IMPORTS, GLOBAL SETTINGS AND UTILS ##

import numpy as np
import pandas as pd
from datetime import timedelta
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
import matplotlib as mpl
from ipywidgets import Layout, VBox, HBox, HTML, interactive_output
from ipywidgets import IntSlider, ToggleButton, Dropdown
from IPython.display import display
import plotly.graph_objects as go

# Global Matplotlib settings
mpl.rcParams['axes.titlesize'] = 12
mpl.rcParams['axes.titleweight'] = 'bold'
mpl.rcParams['axes.labelsize'] = 10
mpl.rcParams['axes.labelweight'] = 'bold'
mpl.rcParams['figure.titlesize'] = 16
mpl.rcParams['figure.titleweight'] = 'bold'
mpl.rcParams['font.family'] = 'DejaVu Sans'
mpl.rcParams['font.sans-serif'] = ['DejaVu Sans']
```

```python
# Custom tick formatter function
def tickfunc(x, pos):
    """
    Custom tick formatter for axis labels.

    Converts large numbers to human-readable format:
        - Billions: 'B'
        - Millions: 'M'
        - Thousands: 'K'
        - Otherwise: integer

    Parameters:
        x (float): Tick value.
        pos (int): Tick position (unused).

    Returns:
        str: Formatted tick label.
    """
    if x >= 1e9: return f'{x/1e9:.1f}B'
    if x >= 1e6: return f'{x/1e6:.1f}M'
    if x >= 1e3: return f'{x/1e3:.1f}K'
    return f'{int(x)}'

# Function to annotate maximum value on a plot
def annotate_max(ax, x_max, y_max, label=None, color='black', h_shift=0, v_shift=0):
    """
    Annotate the maximum value on a matplotlib axis.

    Parameters:
        ax: matplotlib axis
        x_max: x position of the maximum
        y_max: y position of the maximum
        color: arrow color (default: 'black')
        h_shift: horizontal shift for annotation text (default: 0)
        v_shift: vertical shift for annotation text (default: 0)
    """
    if label is None:
        label = f'Max: {y_max:,.2f}'
    ax.annotate(
        label,
        xy=(x_max, y_max),
        xytext=(x_max + h_shift, y_max + v_shift),
        arrowprops=dict(facecolor=color, shrink=0.05, width=1, headwidth=5),
        fontsize=8,
        fontweight='bold',
```

```
        bbox=dict(boxstyle="round,pad=0.3", alpha=0.3, facecolor='lightgrey')
    )
```

1.2 Generazione Dataset

Nella seguente sezione verrà generato un dataset sintetico di vendite per l'analisi con le seguenti feature:

- Order Date (Data dell'ordine)
- Ship Date (Data di spedizione)
- Category (Categoria prodotto)
- Sub-Category (Sottocategoria prodotto)
- Region (Continente)
- State (Stato)
- Price (Prezzo Unitario in €)
- Ravenue (Fatturato Vendita in €)
- Profit (Utile in €)
- Quantity (Quantità prodotti venduti)

[ ]:
```
## DATASET GENERATION ##

print(f"{'='*80}")
print("GENERATING DATASET")
print(f"{'='*80}")

# Create a dictionary mapping regions to countries
region_map = {
    'Europe': ['Italy', 'France', 'Germany', 'Spain', 'United Kingdom'],
    'Americas': ['United States', 'Canada', 'Mexico', 'Brazil', 'Argentina'],
    'Asia': ['China', 'Japan', 'India', 'Russia', 'Indonesia'],
    'Africa': ['South Africa', 'Nigeria', 'Egypt', 'Kenya', 'Morocco'],
    'Oceania': ['Australia', 'New Zealand', 'Papua New Guinea', 'Fiji', 'New␣
 ↪Caledonia']
}

# Create a nested dictionary representing categories, subcategories, products,␣
 ↪and prices
product_map = {
    'Electronics': {
        'Smartphones': {
            'iPhone 14 Pro': 1099.00,
            'Samsung Galaxy S23': 899.00,
            'Google Pixel 7': 649.00,
            'Xiaomi 13': 799.00,
            'OnePlus 11': 729.00
        },
        'Laptops': {
            'MacBook Air M2': 1299.00,
```

```
            'Dell XPS 13': 1149.00,
            'Lenovo ThinkPad X1': 1399.00,
            'HP Pavilion 15': 749.00,
            'ASUS ZenBook 14': 899.00
        }
    },
    'Clothing': {
        'T-Shirts': {
            'Basic cotton T-shirt': 19.99,
            'Ralph Lauren Polo': 89.00,
            'Nike sports T-shirt': 34.99,
            "Levi's vintage T-shirt": 45.00,
            'Lacoste T-shirt': 69.00
        },
        'Jeans': {
            "Levi's 501 Jeans": 98.00,
            'Diesel slim fit Jeans': 159.00,
            'Wrangler regular Jeans': 79.00,
            'Calvin Klein Jeans': 119.00,
            'Lee skinny Jeans': 89.00
        }
    },
    'Home and Kitchen': {
        'Appliances': {
            'Nespresso coffee machine': 189.00,
            'Kenwood food processor': 349.00,
            'Moulinex blender': 59.99,
            'Philips toaster': 39.99,
            'Smeg electric kettle': 149.00
        },
        'Cookware': {
            'Lagostina cookware set': 199.00,
            'Tefal non-stick pan': 45.00,
            'Bialetti pressure cooker': 89.00,
            'Stainless steel wok': 35.00,
            'Le Creuset casserole': 129.00
        }
    },
    'Sport and Fitness': {
        'Sports Shoes': {
            'Nike Air Max running': 139.00,
            'Adidas Ultraboost': 179.00,
            'New Balance 574': 99.00,
            'Asics Gel-Kayano': 159.00,
            'Puma RS-X': 119.00
        },
        'Fitness Equipment': {
```

```
            'Premium yoga mat': 29.99,
            '10kg dumbbell set': 49.00,
            '12kg kettlebell': 35.00,
            'Resistance band': 15.99,
            '5kg medicine ball': 39.00
        }
    }
}

# Create a date range
start='2020-01-01'    # start day
end='2025-12-31'      # end day
dates = pd.date_range(start=start, end=end, freq='D')

# Create DataFrame
dataset = []

for date in dates:
    days_from_start = (date - dates[0]).days
    growth_trend = 1 + (days_from_start / len(dates)) * 0.5
    # 50% growth over the period
    n_sales = int( growth_trend * np.random.poisson(lam=50))
    # Random number of sales per day
    for _ in range(n_sales):
        ship_date = date + timedelta(days=np.random.randint(1, 5))
    # Shipping date 1-4 days after order date
        region = np.random.choice(list(region_map.keys()))
    # Random region
        country = np.random.choice(region_map[region])
    # Random country within the region
        category = np.random.choice(list(product_map.keys()))
    # Random category
        subcategory = np.random.choice(list(product_map[category].keys()))
    # Random subcategory
        product = np.random.choice(list(product_map[category][subcategory].
keys())) # Random product
        price = product_map[category][subcategory][product]
    # Get product price
        quantity = 1 + np.random.poisson(lam=5)
    # Random quantity sold
        revenue = price * quantity
        profit = revenue * np.random.uniform(0.1, 0.3)
    # Profit margin between 10% and 30%

        dataset.append({
            'order_date': date,
```

```
            'ship_date': ship_date,
            'price': price,
            'qty': quantity,
            'revenue': revenue,
            'profit': profit,
            'category': category,
            'sub_category': subcategory,
            'product': product,
            'region': region,
            'state': country,
        })

df = pd.DataFrame(dataset)

# Add randomly missing values
for col in ['region', 'category', 'price']:
    df.loc[df.sample(frac=0.01).index, col] = np.nan

# Display DataFrame info and preview
print("Dataset Info")
print(f"{'-'*80}")
print(df.info(memory_usage='deep'))
print(f"{'-'*80}")
print('Dataset Preview')
print(f"{'-'*80}")
print(df.head().round(2).to_string(index=False))
print(f"{'='*80}")
```

1.3 Pulizia Dati e Downcasting

In questa sezione verrà preparato il dataset prima dell'analisi esplorativa. In particolare verranno eliminati i valori mancanti ed eventuali valori duplicati e infine verrà effettuato il Downcasting per ottimizzare le risorse.

[ ]:
```
## DATASET PREPROCESSING ##

# Remove missing values and duplicates
df.dropna(inplace=True)
df.drop_duplicates(inplace=True)

# Downcasting
for col in df.columns:                          # for each column...
    col_type = df[col].dtype                    # get column type

    if col_type == int:                         # if int...
       df[col] = df[col].astype('int16')        # convert in int32
```

```
    if col_type == float:                          # if float...
        df[col] = df[col].astype('float32')        # convert in float32

    if col_type == object:                         # if object (str)...
        df[col] = df[col].astype('category')       # convert in category

# Extract year from order_date and downcast
df['year'] = df['order_date'].dt.year.astype('Int16')

# Extract month from order_date and downcast
df['month'] = df['order_date'].dt.month.astype('Int16')

print(f"{'='*80}")
print("PREPROCESSED DATASET")
print(f"{'='*80}")
print("Dataset Info")
print(f"{'-'*80}")
print(df.info(memory_usage='deep'))
print(f"{'='*80}")
```

1.4 Analisi Esplorativa

In questa cella vengono creati diversi DataFrame aggregati per supportare l'analisi e la visualizzazione dei dati di vendita:

- **Trend giornaliero**: somma dei profitti e conteggio delle vendite per ogni giorno.
- **Analisi annuale e mensile**: aggregazione di profitti e vendite per anno e per mese.
- **Performance regionale**: aggregazione di profitti e vendite per regione e stato, utile per le mappe geografiche.
- **Performance per sottocategoria**: quantità e profitti aggregati per ogni sottocategoria di prodotto.
- **Calcolo delle medie mobili**: vengono aggiunte colonne con le medie mobili (7, 30, 60 giorni) per analizzare l'andamento di vendite e profitti nel tempo.

Questi dataset aggregati sono la base per le successive visualizzazioni interattive.

[ ]:
```
## DATA ANALYSIS ##

##### Daily trend analysis #####

# Daily aggregation
analysis_trend = df.groupby('order_date', observed=True).agg(
    profit = ('profit', 'sum'),
    sales = ('profit', 'count')
)
```

```
# Moving averages for sales
analysis_trend['MA_7d_sales'] = analysis_trend['sales'].rolling(window=7).mean()
analysis_trend['MA_30d_sales'] = analysis_trend['sales'].rolling(window=30).
 ↪mean()
analysis_trend['MA_60d_sales'] = analysis_trend['sales'].rolling(window=60).
 ↪mean()

# Moving averages for profit
analysis_trend['MA_7d_profit'] = analysis_trend['profit'].rolling(window=7).
 ↪mean()
analysis_trend['MA_30d_profit'] = analysis_trend['profit'].rolling(window=30).
 ↪mean()
analysis_trend['MA_60d_profit'] = analysis_trend['profit'].rolling(window=60).
 ↪mean()

##### Yearly analysis #####

# Yearly aggregation
analysis_yearly = df.groupby('year', observed=True).agg(
    profit = ('profit', 'sum'),
    sales = ('profit', 'count')
)

##### Monthly analysis #####

# Monthly aggregation
analysis_monthly = df.groupby(['year','month'], observed=True).agg(
    profit = ('profit', 'sum'),
    sales = ('profit', 'count')
)

##### Regional analysis #####

# Regional aggregation
analysis_region = df.groupby(['year','region', 'state'], observed=True).agg(
    profit = ('profit', 'sum'),
    sales = ('profit', 'count')
).reset_index()


##### Product performance analysis #####

# Product aggregation
analysis_product = df.groupby('sub_category', observed=True).agg(
    quantity = ('qty', 'sum'),
    profit = ('profit', 'sum')
)
```

```
# Profit Per Unit (PPU)
analysis_product['PPU'] = analysis_product['profit'] /␣
 ↪analysis_product['quantity']
```

1.5 Visualizzazioni Interattive

In questa sezione vengono presentati diversi strumenti interattivi per esplorare il dataset di vendite. Attraverso dashboard dinamiche e mappe geografiche, è possibile analizzare l'andamento di vendite e profitti nel tempo, confrontare le performance tra anni e regioni, e individuare facilmente i valori massimi e le tendenze principali.

- **Product Performance**: Una dashboard interattiva consente di analizzare le performance delle sottocategorie di prodotto. L'utente può selezionare la metrica di interesse (Profitto, Quantità, Profitto per Unità) e il numero di sottocategorie da visualizzare. Il grafico a barre orizzontali mostra le sottocategorie con i valori più alti per la metrica scelta, facilitando l'individuazione dei prodotti più rilevanti.
- **Sales & Profit Analysis**: Una dashboard interattiva con quattro grafici (vendite e profitti annuali e mensili). Un selettore consente di scegliere l'anno e aggiornare i grafici mensili, con annotazione automatica del valore massimo.
- **Sales & Profit Trend**: Si visualizzano i trend giornalieri di vendite e profitti, con la possibilità di attivare/disattivare le medie mobili (7, 30, 60 giorni) tramite pulsanti toggle. Questo permette di analizzare l'andamento nel tempo e confrontare i dati reali con le medie mobili.
- **Regional Performance**: Si visualizza una mappa geografica interattiva che mostra vendite e profitti per stato e regione, suddivisi per anno. Un selettore consente di visualizzare i dati di anni diversi. I marker sulla mappa sono dimensionati in base alle vendite e, passando il mouse, si visualizzano dettagli su stato, vendite e profitti, oltre ai totali regionali.

Queste visualizzazioni permettono di esplorare in modo efficace l'evoluzione delle vendite e dei profitti nel tempo e nello spazio.

[ ]:
```
## INTERACTIVE VISUALIZATION - PRODUCT PERFORMANCE ##

# Widget backend for interactive plots
%matplotlib widget

# Turn off interactive mode
plt.ioff()

# Define available metrics
metrics = {
    'Profit': 'profit',
    'Quantity': 'quantity',
    'Profit Per Unit': 'PPU'
}

# Update plot function
```

```
def plot_product_performance(metric, top_n):

    # Select metric column
    metric_col = metrics[metric]

    # Reord and take top N
    data_sorted = analysis_product.sort_values(by=metric_col, ascending=True).
 ↪tail(top_n)

    # Create figure
    fig, ax = plt.subplots(figsize=(9, 6))

    # Set figure for widget display
    fig.canvas.header_visible = False
    fig.canvas.layout.width = '900px'
    fig.canvas.layout.height = '600px'

    # Horizontal bars
    bars = ax.barh(
        data_sorted.index,
        data_sorted[metric_col],
        color='steelblue', edgecolor='black',
        alpha=0.7
    )

    # Add annotations to bars
    for bar in bars:
        width = bar.get_width()
        ax.annotate(
            tickfunc(width, None),
            xy=(width, bar.get_y() + bar.get_height() / 2),
            xytext=(5, 0),  # 5 points horizontal offset
            textcoords="offset points",
            ha='left',
            va='center',
            fontsize=8,
            fontweight='bold'
        )

    # Calculate axis limits
    x_max = data_sorted[metric_col].max()
    x_min = data_sorted[metric_col].min()
    diff = x_max - x_min

    # Axis formatting and labels
    ax.set_xlim(max([x_min - 0.2*diff, 0]), x_max + 0.2*diff)
    ax.set_xlabel(metric)
```

```
    ax.set_ylabel('Sub-Category')
    ax.set_title(f'Top {top_n} Sub-Categories by {metric}')
    ax.grid(axis='x', linestyle='--', alpha=0.3)

    if metric_col in metrics.values():
        ax.xaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))

    plt.tight_layout()
    plt.show()

# Widget Dropdown for metrics
dropdown_metric = Dropdown(
    options=list(metrics.keys()),
    value='Profit',
    description='Metric:',
    style={'description_width': 'initial'}
)

# Widget Slider for top N products
slider_top_n = IntSlider(
    value=10,
    min=5,
    max=10,
    step=1,
    description='Top N:',
    continuous_update=False,
    style={'description_width': 'initial'},
    layout=Layout(width='80%', height='2%')

)

# Interactive output
output = interactive_output(
    plot_product_performance,
    {
        'metric': dropdown_metric,
        'top_n': slider_top_n
    }
)

# Final layout
dropdown = VBox([dropdown_metric])
slider = HBox([slider_top_n], layout=Layout(justify_content='flex-end'))
ui_product = VBox([dropdown, output, slider])
display(ui_product)
```

[ ]:
```
## INTERACTIVE VISUALIZATION - SALES/PROFIT OVER YEARS ##

# Widget backend for interactive plots
%matplotlib widget

# Turn off interactive mode
plt.ioff()

# Years available
years = analysis_monthly.index.get_level_values('year').unique().sort_values()

# Create the figure
fig, ax = plt.subplots(2, 2, figsize=(9, 6))
fig.suptitle('Sales and Profit Analysis')

# Set figure for widget display
fig.canvas.header_visible = False
fig.canvas.layout.width = '900px'
fig.canvas.layout.height = '600px'

# Limits for yearly plots
max_sales = analysis_yearly['sales'].max()
max_profit = analysis_yearly['profit'].max()

# Chart 1: Yearly Sales (static)
ax[0,0].bar(
    analysis_yearly.index, analysis_yearly['sales'],
    label='Sales',
    color='steelblue', edgecolor='black', alpha=0.7
)

# Annotate max value
annotate_max(
    ax[0,0],
    x_max=analysis_yearly['sales'].idxmax(),
    y_max=analysis_yearly['sales'].max(),
    h_shift=-3,
    v_shift= max_sales * 0.15
)

# Axis formatting and labels
ax[0,0].yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))
ax[0,0].set_ylabel('Sales Amount')
ax[0,0].set_xlabel('Year')
ax[0,0].set_ylim(0, max_sales * 1.3)
ax[0,0].set_title('Yearly Sales')
ax[0,0].grid(axis='y', linestyle='--', alpha=0.5)
```

```
# Chart 2: Yearly Profit (static)
ax[0,1].bar(
    analysis_yearly.index, analysis_yearly['profit'],
    label='Profit',
    color='coral', edgecolor='black', alpha=0.7
)

# Annotate max value
annotate_max(
    ax[0,1],
    x_max=analysis_yearly['profit'].idxmax(),
    y_max=analysis_yearly['profit'].max(),
    h_shift=-3,
    v_shift= max_profit * 0.15
)

# Axis formatting and labels
ax[0,1].yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))
ax[0,1].set_ylabel('Profit Amount')
ax[0,1].set_xlabel('Year')
ax[0,1].set_ylim(0, max_profit * 1.3)
ax[0,1].set_title('Yearly Profit')
ax[0,1].grid(axis='y', linestyle='--', alpha=0.5)

# Initialize monthly charts with the first year
initial_year = years[0]
sales_data = analysis_monthly.loc[initial_year]['sales']
profit_data = analysis_monthly.loc[initial_year]['profit']
months = analysis_monthly.loc[initial_year].index

# Limits for monthly plots
max_sales = analysis_monthly['sales'].max()
max_profit = analysis_monthly['profit'].max()
max_sales_idx = sales_data.idxmax()
max_profit_idx = profit_data.idxmax()

# Chart 3: Monthly Sales (save reference to bars)
bars_sales = ax[1,0].bar(
    months, sales_data,
    label='Sales',
    color='steelblue', edgecolor='black', alpha=0.7
    )

# Axis formatting and labels
ax[1,0].yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))
ax[1,0].set_ylabel('Amount')
```

```
ax[1,0].set_xlabel('Month')
ax[1,0].set_ylim(0, max_sales * 1.3)
title_sales = ax[1,0].set_title(f'Monthly Sales - {initial_year}')
ax[1,0].grid(axis='y', linestyle='--', alpha=0.5)
ax[1,0].set_xticks(months)

# Chart 4: Monthly Profit (save reference to bars)
bars_profit = ax[1,1].bar(
    months, profit_data,
    label='Profit',
    color='coral', edgecolor='black', alpha=0.7
    )

# Axis formatting and labels
ax[1,1].yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))
ax[1,1].set_ylabel('Amount')
ax[1,1].set_xlabel('Month')
ax[1,1].set_ylim(0, max_profit * 1.3)
title_profit = ax[1,1].set_title(f'Monthly Profit - {initial_year}')
ax[1,1].grid(axis='y', linestyle='--', alpha=0.5)
ax[1,1].set_xticks(months)

plt.tight_layout()

# Function to update monthly plots based on selected year
def update_plots(year):
    # Extract new data
    sales_data = analysis_monthly.loc[year]['sales']
    profit_data = analysis_monthly.loc[year]['profit']

    max_sales_idx = sales_data.idxmax()
    max_profit_idx = profit_data.idxmax()
    max_sales = sales_data.max()
    max_profit = profit_data.max()

    # Update heights of Sales bars
    for bar, height in zip(bars_sales, sales_data):
        bar.set_height(height)

    # Update heights of Profit bars
    for bar, height in zip(bars_profit, profit_data):
        bar.set_height(height)

    # Update titles
    ax[1,0].set_title(f'Monthly Sales - {year}')
    ax[1,1].set_title(f'Monthly Profit - {year}')
```

```
    # Remove previous annotations
    for annot in ax[1,0].texts + ax[1,1].texts:
        annot.remove()

    # Horizontal shift based max position
    if max_sales_idx >= months.max()/2:
        h_shift = -5
    else:
        h_shift = 2

    # Add new annotations
    annotate_max(
        ax[1,0],
        x_max=max_sales_idx,
        y_max=max_sales,
        h_shift=h_shift,
        v_shift= max_sales * 0.15
    )

    # Horizontal shift based on position
    if max_profit_idx > months.max()/2:
        h_shift = -5
    else:
        h_shift = 2

    # Add new annotations
    annotate_max(
        ax[1,1],
        x_max=max_profit_idx,
        y_max=max_profit,
        h_shift=h_shift,
        v_shift= max_profit * 0.15
    )

    # Redraw only the modified parts
    fig.canvas.draw_idle()

# Create slider
year_slider = IntSlider(
    value=initial_year,
    min=years.min(),
    max=years.max(),
    step=1,
    description='Year:',
    continuous_update=True,
    style={'description_width': 'initial'},
    layout=Layout(width='80%', height='2%')
```

```
)

# Connect the slider to the update function
output = interactive_output(update_plots, {'year': year_slider})

# Final layout
slider = HBox([year_slider], layout=Layout(justify_content='center'))
ui_year = VBox([fig.canvas, slider])
display(ui_year)
```

[ ]:
```
## INTERACTIVE VISUALIZATION - SALES/PROFIT TRENDS ##

# Widget backend for interactive plots
%matplotlib widget

# Turn off interactive mode
plt.ioff()

# Create the figure
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(9, 6))
fig.suptitle('Sales and Profit Trend with Moving Averages')

# Set figure for widget display
fig.canvas.header_visible = False
fig.canvas.layout.width = '900px'
fig.canvas.layout.height = '600px'

##### Principal lines - Daily Trend (visible) #####

# Sales trend line
line_sales, = ax1.plot(
    analysis_trend.index, analysis_trend['sales'],
    label='Sales', color='skyblue',
    linewidth=1, linestyle='--', alpha=0.5
    )
# Profit trend line
line_profit, = ax2.plot(
    analysis_trend.index, analysis_trend['profit'],
    label='Profit', color='skyblue',
    linewidth=1, linestyle='--', alpha=0.5
    )

##### Moving average lines (hidden) #####

## SALES ---

# MA 7 days - sales
```

```
line_ma7_sales, = ax1.plot(
    analysis_trend.index, analysis_trend['MA_7d_sales'],
    label='MA 7d', color='olive', linewidth=2,
    visible=False
    )

# MA 30 days - sales
line_ma30_sales, = ax1.plot(
    analysis_trend.index, analysis_trend['MA_30d_sales'],
    label='MA 30d', color='coral', linewidth=2,
    visible=False
    )

# MA 60 days - sales
line_ma60_sales, = ax1.plot(
    analysis_trend.index, analysis_trend['MA_60d_sales'],
    label='MA 60d', color='navy', linewidth=2,
    visible=False
    )

## PROFIT ---

# MA 7 days - profit
line_ma7_profit, = ax2.plot(
    analysis_trend.index, analysis_trend['MA_7d_profit'],
    label='MA 7d', color='olive', linewidth=2,
    visible=False
    )

# MA 30 days - profit
line_ma30_profit, = ax2.plot(
    analysis_trend.index, analysis_trend['MA_30d_profit'],
    label='MA 30d', color='coral', linewidth=2,
    visible=False
    )

# MA 60 days - profit
line_ma60_profit, = ax2.plot(
    analysis_trend.index, analysis_trend['MA_60d_profit'],
    label='MA 60d', color='navy', linewidth=2,
    visible=False
    )

# Axis formatting and labels
ax1.set_ylabel('Sales Amount')
ax1.set_title('Daily Sales Trend')
ax1.grid(True, linestyle='--', alpha=0.3)
```

```
ax1.legend(loc='best')
ax1.yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))

ax2.set_ylabel('Profit Amount')
ax2.set_xlabel('Date')
ax2.set_title('Daily Profit Trend')
ax2.grid(True, linestyle='--', alpha=0.3)
ax2.legend(loc='best')
ax2.yaxis.set_major_formatter(ticker.FuncFormatter(tickfunc))

plt.tight_layout()

# Dictionary to map buttons to lines #####
ma_lines = {
    'MA 7d': (line_ma7_sales, line_ma7_profit),
    'MA 30d': (line_ma30_sales, line_ma30_profit),
    'MA 60d': (line_ma60_sales, line_ma60_profit)
}

# Toggle moving averages function
def toggle_ma(change, ma_name):

    # Get corresponding lines
    sales_line, profit_line = ma_lines[ma_name]

    # Set visibility based on toggle state
    sales_line.set_visible(change['new'])
    profit_line.set_visible(change['new'])

    # Update legends
    ax1.legend(loc='best')
    ax2.legend(loc='best')

    # Redraw only the modified parts
    fig.canvas.draw_idle()


##### Create toggle buttons #####

# MA 7 days
btn_ma7 = ToggleButton(
    value=False,
    description='MA 7 days',
    button_style='primary',
    icon='eye'
)
```

```
# MA 30 days
btn_ma30 = ToggleButton(
    value=False,
    description='MA 30 days',
    button_style='primary',
    icon='eye'
)

# MA 60 days
btn_ma60 = ToggleButton(
    value=False,
    description='MA 60 days',
    button_style='primary',
    icon='eye'
)

##### Layout buttons #####

# Buttons title layout
title = HTML(value="<b>Toggle Moving Averages</b>")
title_buttons = HBox(
    [title],
    layout=Layout(justify_content='center', width='100%')
    )

# Buttons layout
buttons = HBox(
    [btn_ma7, btn_ma30, btn_ma60],
    layout=Layout(justify_content='center', width='100%')
    )


# Connect buttons to functions
btn_ma7.observe(lambda change: toggle_ma(change, 'MA 7d'), names='value')
btn_ma30.observe(lambda change: toggle_ma(change, 'MA 30d'), names='value')
btn_ma60.observe(lambda change: toggle_ma(change, 'MA 60d'), names='value')

# Final layout
ui_trend = VBox([fig.canvas, title_buttons, buttons])
display(ui_trend)
```

[ ]:
```
## INTERACTIVE VISUALIZATION - REGIONAL PERFORMANCE ##

# Get years and regions
years = sorted(analysis_region['year'].unique())
regions = analysis_region['region'].unique()
```

```
# Create the figure
fig_geo = go.Figure()

# Create traces for each year-region combination
for year in years:
    # Filter data for the year
    df_year = analysis_region[analysis_region['year'] == year]
    for region in regions:
        # Filter data for the region
        df_region = df_year[df_year['region'] == region]
        if len(df_region) > 0:
            # Add scattergeo trace
            fig_geo.add_trace(go.Scattergeo(
                locations=df_region['state'],
                locationmode='country names',
                text=[f"{state}" for state in df_region['state']],
                marker=dict(
                    size=df_region['sales'] /  25,
                    line=dict(width=1, color='white'),
                    sizemode='diameter'
                ),
                name=region,
                legendgroup=region,
                showlegend=True,
                visible=(year == years[0]),
                customdata = np.stack(
                    [
                    df_region['profit'],
                    df_region['sales'],
                    np.full(len(df_region), df_region['sales'].sum()),
                    np.full(len(df_region), df_region['profit'].sum()),
                    np.full(len(df_region), region)
                    ],
                    axis=-1
                ),
                hovertemplate=(
                    "<b>%{text}</b><br>"
                    "Sales %{customdata[1]}<br>"
                    "Profit: €%{customdata[0]:,.2f}<br>"
                    "<extra><b>%{customdata[4]}</b><br>Sales: %{customdata[2]:
 ↪,}<br>Profit: €%{customdata[3]:,.0f}</extra>"
                ),
            ))

# Create slider steps
steps = []
for i, year in enumerate(years):
```

```
    # Each year has len(regions) traces
    visibility =(
        [False] * i * len(regions) +
        [True] * len(regions) +
        [False] * (len(years)-i-1) * len(regions)
    )
    step = dict(
        method='update',
        args=[{'visible': visibility}],
        label=str(year)
    )
    steps.append(step)

# Update layout with slider and geo settings
fig_geo.update_layout(
    sliders=[dict(
        active=0,
        currentvalue={
            "prefix": "Year: ",
            "font": {"size": 16}
        },
        #pad={"t": 5, "b": 0},
        len=0.6,
        x=0.2,

        steps=steps
    )],
    geo=dict(
        projection_type='winkel tripel',
        showocean=True,
        oceancolor='skyblue',
        landcolor='black',
    ),
    template='plotly_dark',
    title={
        'text': 'Sales & Profit by State & Region Over Years',
        'x': 0.5,
        'xanchor': 'center',
        'font': {'size': 24}
    },
    legend={
        'title': {'text': 'Regions', 'font': {'size': 16}},
        'orientation': 'v',
        'x': 1.02,
        'y': 1
    },
    height=600,
```

```
    width=900
)

fig_geo.show()
```